

Steps for RAG Ingestion

A step-by-step guide to setting up Retrieval-Augmented Generation (RAG) ingestion using MongoDB Atlas, Mistral AI embeddings, and a Node.js backend pipeline.

A l w a y s A h e a d

Step 1: Database & Network Access

Configure your MongoDB Atlas database user with the appropriate privileges to enable secure access for the RAG pipeline.

01

Open Database & Network Access

Navigate to **Database & Network Access** from the left side panel in MongoDB Atlas.

03

Set Privileges

Under **Database User Privileges**, add the built-in role as **Atlas Admin**.

02

Add New Database User

Click **Add New Database User** and enter your desired **username** and **password**.

04

Confirm & Save

Click **Add User** to finalize and save the new database user configuration.

Step 2: Create Database & Collection

Navigate to Data Explorer

Go to **Data Explorer** from the left side panel in your MongoDB Atlas dashboard.

Connect to Your Cluster

Click on the **Connect** option on the name of your cluster to open the connection options.

Create Database & Collection

Click the **+** icon to create a new database. Enter your desired **database name** and **collection name**, then click **Create Database** to confirm.

Step 3: Create Vector Search Index

Set up the vector search index in MongoDB Atlas to enable semantic similarity search over your embedded test case data.

1

Open Search & Vector Search

Go to **Search and Vector Search** from the left side panel. Click **Create Search Index**.

2

Configure Index Type

Select **Vector Search** as the search type. Choose **Bring Your Own Embeddings**. Enter the index name: `vector_index_test_cases`.

3

Select Database & Collection

Select database `RagTest_cases` and collection `test_cases`. Set configuration method to **JSON Editor**.

4

Enter Index Definition

Delete existing JSON fields and paste the index definition below. Click **Next** and then **Create Search Index**. The index will be ready in a few minutes.

```
{
  "fields": [
    { "type": "vector", "path": "embedding", "numDimensions": 1024, "similarity": "cosine" },
    { "type": "filter", "path": "id" },
    { "type": "filter", "path": "module" },
    { "type": "filter", "path": "title" },
    { "type": "filter", "path": "description" },
    { "type": "filter", "path": "steps" },
    { "type": "filter", "path": "expectedResults" }
  ]
}
```

 After a few minutes, the index status will change to **Active**. Wait for this before proceeding to the next step.

Step 4: Get the Atlas Connection String

Retrieve your MongoDB Atlas connection string to use in your application's environment configuration.

→ Open Cluster Connection

In Atlas, open your **Cluster** and navigate to **Connect → Drivers**.

→ Copy the Connection String

Click **Copy** to copy the connection string template. It will look like:

```
mongodb+srv://<username>:<password>@<cluster>/?retryWrites=true&w=majority&appName=<appName>
```

→ Replace Placeholder Values

Replace `<username>` and `<password>` with your actual credentials. Ensure the password is **URL-encoded** to avoid connection issues.

⚠ Always URL-encode special characters in your password (e.g., `@` becomes `%40`). Failure to do so will result in connection errors.

Step 5: Configure the .env File

Open your application's `.env` file and update the following environment variables with your MongoDB and Mistral AI credentials.

```
MONGODB_URI="Connection string to be entered here"  
DB_NAME="<your_database_name>"  
COLLECTION_NAME="<your_collection_name>"  
VECTOR_INDEX_NAME="<your_vector_index_name>"
```

```
# Mistral AI Configuration (for embeddings)
```

```
MISTRAL_API_KEY=<your_mistral_api_key>  
MISTRAL_EMBEDDING_MODEL="mistral-embed"
```

MONGODB_URI

Your full Atlas connection string with credentials.

DB_NAME

The name of your MongoDB database (e.g., `RagTest_cases`).

MISTRAL_API_KEY

Your Mistral AI API key for generating embeddings.

❏ Keep your `.env` file out of version control. Add it to your `.gitignore` to prevent accidental exposure of credentials.

Step 6: Install Dependencies

Before running any scripts, install all required Node.js dependencies from the **root folder** of your project.

Run the following command:

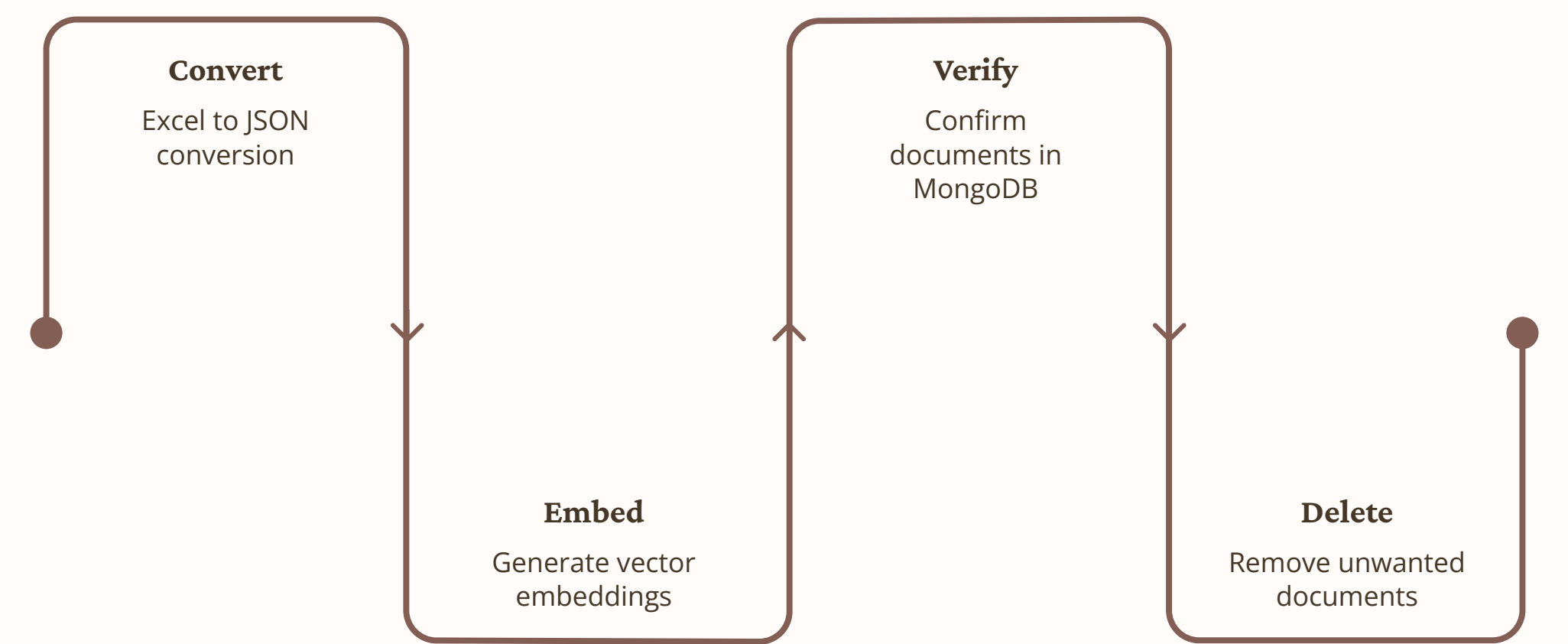
```
npm install
```

- ✅ This will install all packages listed in `package.json`, including MongoDB drivers, Mistral AI SDK, and utility libraries needed for the ingestion pipeline.

Ensure you are in the **root directory** of the project before running this command. If you encounter permission errors, try running with elevated privileges or check your Node.js version compatibility.

Step 7: Convert Data & Create Embeddings

Run the following scripts in sequence to convert your Excel data to JSON, generate embeddings, store them in MongoDB, and verify the ingestion.



Each script below corresponds to a stage in the pipeline above. Run them in order for a successful ingestion.

1. Convert Excel to JSON

```
node src/scripts/data-conversion/excel-to-json.js
```

Transforms your Excel test case file into a structured JSON format ready for embedding.

2. Create Embeddings & Store Data

```
node src/scripts/embeddings/create-embeddings-batch-mistral.js
```

Generates vector embeddings using Mistral AI and stores the documents in your MongoDB collection.

3. Verify in MongoDB

Once the command completes, open **MongoDB Atlas** and navigate to your collection. Confirm the documents are visible and correctly structured.

4. Delete All Documents

```
node src/scripts/utilities/delete-all-documents.js
```

After verifying the ingestion, run this cleanup script to remove all documents from the collection before the final Postman-based workflow.

Step 8: Postman API Verification

Once documents are successfully deleted, open **Postman** and import the shared collection. Run the following requests in order to validate the full RAG ingestion pipeline for test cases.



1. Health Check

Verify the backend service is running and reachable.



2. Upload Excel – Test Cases

Upload the Excel file for test cases. The file is located at:
`rag-mongo-demo-backend\src\data`



4. List Available Files

Confirm the uploaded Excel file is listed and available for processing.



5. Create Embeddings – Batch (Fast)

Trigger the batch embedding creation for test cases using the fast processing mode.

- ✔ Once all Postman requests complete successfully, your RAG ingestion pipeline is fully operational and ready for semantic search queries over your test case data.